CS673 Software Engineering

Team 3 - SpecCheck

Project Proposal and Planning

Your project Logo here if any

Team Member	Role(s)	Signature	Date
Danielle Leask	Team Lead & Requirement Lead	*Danielle Leask*	5/13/26
Vy Trinh	Design and implementation lead & Security Lead	*Vy Trinh*	5/13/2026
Jannatul Tuba	Configuration Lead &QA Lead	Jannatul Tuba	5/13/26
Yuting Shi	Design and implementation lead & Configuration Lead	*yuting shi*	5/17/2026

Revision history

Version	Author	Date	Change
0.0	**Danielle Leask**	**5/10/2026**	Updated Roles, Overview of Project, added risk management link
0.0	**Jannatul Tuba**	**5/11/2026**	Added Related work and Functional Requirements
0.0	**Danielle Leask**	**5/12/2026**	Updated configuration management plan and quality assurance plan, including metrics and code review process.
0.1	**Vy Trinh**	**5/12/2026**	Updated essential feature, nonfunctional requirements, management plan

			(priorities) and testing
0.0	**Jannatul Tuba**	**5/13/2026**	Updated with details for Code Commit Guideline and Git Branching Strategy, Testing (timeline,assignee,type) , Defect Management(severity)

1. Overview

Motivation: Help software companies reduce the time and manual effort spent creating test cases and validating product features.

Purpose: Automatically generate functional, negative, and edge case test cases from product documentation to improve software quality and provide early feedback on features.

Potential Users: Software companies, including developers, QA engineers, and product teams.

Basic Functionality: Users upload product documents (PDFs, API specs, etc.), and the system uses AI to analyze them and generate test cases from a given user story.

Technology Stack: The system will be built using Django for the backend and web interface, MongoDB as the database, and Jira for agile project management. AI/ML

tools will be integrated to enable automated document processing and natural language understanding for test case generation.

2. Related Work

TestSigma, TestRail, and Katalon Studio can generate test cases based on a user story, but the biggest gap is that these tools do not have an option to provide existing product knowledge as context or upload additional documents to let the AI understand the full context of the product. The generated test cases therefore tend to be generic and often miss existing product validation rules, known edge cases, or business logic that only exists inside internal documentation. There is also no way to build on that context over time, meaning every session starts from scratch.

Our goal is to develop a web-based application where users can provide context in the form of PDFs, API specs, and feature documents, and then have the AI generate targeted test cases when a user story is submitted. The system stores the uploaded knowledge persistently using a vector database so that it can be retrieved and used as grounded context at generation time, rather than relying on the AI to guess at product behavior it has never seen.

3. Proposed High level Requirements

Functional Requirements:

- **Essential Features:**
    - **Login and Registration:** As a user, I want to create an account and log into the system securely so that I can manage my uploaded documents and generated test cases.

      User Story**:** As a user, I want to create an account and log in securely so that I can manage my own uploaded documents and generated test cases.

      Roles**:** The system handles three distinct roles to manage team workflows. A QA Engineer can manage files and generate test cases. A QA Lead inherits all QA Engineer permissions and has exclusive access to impact analysis dashboards. A Developer has view only access to check finalized test cases attached to their assigned tasks before writing code.

    - **Document Upload and Knowledge Indexing** : As a QA, I want to upload pdf and image file so that the system has product context

      User Story: As a QA Engineer, I want to upload product documents like PDF files and images in batches so that the system can index them into a global knowledge base.

Constraints: Users can upload up to 10 files in a single batch with a maximum file size of 20MB per file. The ingestion pipeline accepts PDF files for business requirements and standard images like PNG or JPEG for user interface designs or mockups. All uploaded files form a project wide global knowledge base instead of being tied to one specific task.

- **User Story Input and Analysis:** As a QA Engineer, I want to provide product user story and have AI retrieve product information automatically so that it can generate test case while having product context

  User Story: As a QA Engineer, I want to input a specific user story manually so that the AI can automatically retrieve relevant context from the global knowledge base.

  Ingestion Workflow: Users must type or paste each user story manually and separately. The application does not automatically scan raw text files to extract user stories to give the QA Engineer absolute control over the target requirements. The application uses a Retrieval Augmented Generation pipeline. Product documents are processed, embedded, and saved into a vector database. When a user story is processed, the system automatically fetches the exact technical context from this global data pool to prevent generic outputs.

- **AI Test Case Generation :** As a QA Engineer, I want AI to generate functional, positive, negative, edge cases test cases with steps based on the user story while considering existing product context

  User Story: As a QA Engineer, I want the AI to generate functional, positive, negative, and edge case test cases with detailed steps based on my configuration and instructions.

  Quality Control: The application uses existing state of the art AI model APIs like GPT 4o or Claude 3.5 Sonnet instead of training a specialized model from scratch. Creating test cases is an interactive process rather than a single click action. Users can configure the output by choosing specific test categories and entering custom prompt guidelines like behavior driven development templates. Every generation request creates between 5 to 15 targeted test cases. The system ensures high quality outputs through a Human in the Loop review where users can edit text, reject low quality points, or approve them before saving. Additionally, a secondary evaluation AI agent

checks the generated test cases against the original product guidelines to score compliance and flag missing edge cases.

- **Document Management Dashboard** : As a QA Engineer I want to view, add, update, delete documents uploaded in for knowledge

  User Story: As a QA Engineer, I want to view, add, update, and delete uploaded documents in a centralized dashboard.

  Data Scope: Changes made in this dashboard update the vector database in real time. Deleting a document removes its text embeddings from the global knowledge base immediately, ensuring the AI model no longer uses outdated data for future test case generation cycles.

- **Export Test Case :** As QA, I want to export generated test cases as a text or CSV file, so that I can import it in other test management system

  User Story: As a QA Engineer, I want to export generated test cases as Text or CSV files so that I can easily import them into other test management systems.

  Formatting: The system maps the generated test case fields like title, steps, expected results, and test type into standard CSV columns. This clean file structure allows direct integration with popular third party agile and test management tools.

- **Desired Features:**
  - **Impacted Feature Highlighting :** As QA lead I want the system to list all impacted feature based on the user story

    User Story: As a QA Lead, I want the system to analyze the new user story against existing documents so that it can identify and list which existing system features might be affected by this new change.

    Regression Analysis: Impacted features represent the active regression testing scope. When a user introduces a new user story, the system highlights which existing software modules might break because of this update. This alert is triggered by analyzing changes against the global

product documents stored in the vector database, allowing the QA Lead to plan proper regression testing. Only users with the QA Lead role can access this analytics view.

- Test case history: As a user, I want the system to save previously generated test cases so that I can review and reuse them later

  User Story: As a user, I want the system to save previously generated test cases so that I can review, edit, and reuse them later.

  Storage: Approved test cases are stored in a relational database linked to the specific user account and project ID. This history log maintains version control, allowing users to see how test cases evolve when a user story is updated.

- **Optional Features:**
  - **Github PR analyzer:** As QA, I want the system to automatically analyze a pull request and to analysis based on the code change and suggest test cases based on the PR code change

    User Story: As a QA Engineer, I want the system to automatically analyze GitHub Pull Requests and connect with Agile tools like Jira so that it can suggest test cases directly inside our project tasks based on the code changes.

    Integration: Instead of acting as a standalone tool, this feature functions as a plugin or webhook integration. When a developer creates a Pull Request on GitHub, the system triggers the AI pipeline to analyze the code diff, references the global product documents, and pushes the newly suggested test cases directly into the corresponding Jira ticket.

Nonfunctional Requirements:

- The system should generate test cases within a reasonable response time

The system must process requests efficiently to maintain a smooth user workflow. Standard user interface actions like dashboard navigation must respond within 2 seconds. The retrieval and indexing of an uploaded document must complete within 10 seconds per file. The AI test case generation pipeline using the external model API must deliver the final structured test cases within 15 seconds per request.

- The application should support secure authentication and protected user data
- Uploaded documents should be processed reliably and stored securely
- The system should be scalable to support multiple users simultaneously
- The UI should be responsive and user-friendly
- AI-generated outputs should be clear, structured, and easy to understand

4. Management Plan

a. Objectives and Priorities

This project focuses on integrating AI-driven test case generation into existing applications to streamline the testing process, reduce manual effort, and enable teams to dedicate more time to building and enhancing new features.

The highest priority objectives are:

- Build a reliable document upload and processing system for PDFs, API specifications, and product documents
- Integrate AI models to generate accurate functional, negative, and edge-case test cases
- Ensure generated test cases are context-aware and based on uploaded product knowledge
- Create a clean and user-friendly dashboard for managing uploaded documents and generated test cases
- Maintain system performance, security and scalability

b. Risk Management (need to be updated constantly)

The highest priority risks are mainly around system design, testing, and technology competence. If the architecture isn't planned properly from the start, the system could become messy and hard to scale. This will cause a bigger issue when connecting the AI with document processing. To avoid that, the team should agree on a clear design early on and keep reviewing it as features are added. Testing is also a major risk because if we don't test continuously, bugs can build up and the AI-generated outputs may not be reliable or accurate. This can be prevented by testing each feature as it is built instead of waiting until the end. Technology competence is another concern since not everyone is confident in the same programming language and we also do not have experience yet with how to implement the AI component of the project. We plan to split tasks based on individual strengths, learn the required tools together through documentation and tutorials, and rely on regular team check-ins to support each other as we build the system.

Risk management link:
https://docs.google.com/spreadsheets/d/1TGf5X4D6LBQliZie8Sje-MLOzPqzJq3oUjNYyLj5sdw/edit?usp=sharing

Risk Management Sheet Link:

c. Timeline (this section should be filled in iteration 0 and updated at the end of each later iteration)

Iteration	Functional Requirements(Essential/ Disable/Option)	Tasks (Cross requirements tasks)	Estimated/real total person hours
1	User authentication, document upload, dashboard	Set up GitHub repository, Jira, CI/CD, UML design, set up frontend, backend architecture, pdf & document upload,framework build	Estimated: 40 hours Real:
2	User story analysis and test case generator	Integrate open AI,claud api, process user story	Estimated: 60 hours Real:
3	Export Test cases, validation deployment, history, improve UI	Download feature, save generated test case, improve UI, error handling, refactor, integration test.	Estimated: 40 hours Real:

5. Configuration Management Plan

a. Tools

- Language backend: Python
- Front end: Javascript, CSS, HTML
- Framework: Django
- DB: MongoDB

- Ai Tool: OpenAI/Claude
- IDE: VS Code
- DevOps: Github
- Project Management: Jira
- Testing: Playwright, pytest

b. Code Commit Guideline and Git Branching Strategy

Branching Strategy:

- Main: Production version. Only merged at the end of each iteration for demo. Never commit directly
- Development:Integration purpose. All feature branches merge here via pull request. Must always be in working state
- feature (each feature): One branch per Jira user story. It should be branched from develop, merged back via pull request. Close after merge
- **bug fix branch:** For bug fixes found during testing.Similar pattern as feature branch

Naming Convention for Branch

- main
- develop
- feature/document-upload
- bugfix/pdf-parse-empty-file

Code Commit Guideline:

- Commit descriptions should be clear and simple.
- Every comment should start with either [HUMAN],[AI],[HYBRID]

Quality Assurance Plan

c. Metrics

Metric Name	Description
Lines of Code	Total number of lines for backend and frontend to track project growth.
# of generated test cases	Number of test cases from users story Measures how many functional, negative, and edge-case test cases are generated from a user story
Defect rate	Number of bugs logged for iteration

Test case accuracy	Measures whether generated test cases correctly reflect the uploaded product documentation
Test pass rate	Integration test, unit test,function test
personHour for project	140 person hours
# of user stories	13 user stories

d. CI/CD Plan

We will use a shared repository and every feature will need to go through a pull request and be tested in GitHub Action before merging to the main branch. If it fails the merge will be blocked. We will be using Django, this will allow us to keep deploying on the web. We will deploy the application on a BU private domain.

e. Coding Standard

We will be using Google's coding standards for HTML/CSS and for Python.
https://google.github.io/styleguide/htmlcssguide.html
Google Python Style Guide
A lot of these standards are very common in industry and are good habits to learn, and also will make our code more understandable to each other. The commenting standards cover what is important about each class/function/method/etc.

f. Code Review Process

We will be tracking which documents have been reviewed by which team members to make sure that everybody is on the same page. The design/implementation leaders (we have 2 members) should be making sure that each new additional piece of code will fit into the existing infrastructure, or deciding if the existing infrastructure should be changed to allow for updates. We will be adding detailed comments to each commit, and using pull requests for merging separate branches.

g. Testing

The project will include both manual and automated testing to ensure system reliability and software quality.
+Manual Testing:
Team members will manually test:

- User login and registration
- PDF and document uploads
- AI-generated test cases
- Dashboard and export functionality

- Error handling and invalid inputs

+Automated Testing:

- PlayWright python for automated testing
- PyTest will be used for backend unit testing
- GitHub Actions will automatically run tests during pull requests and deployments

+Testing Objectives:

-Verify uploaded documents are processed correctly
-Detect bugs early during development
-Prevent integration failures between frontend, backend, database, and AI
-Validate system performance and stability

Type	**Tool**	**Assignee**	**When**
Unit Test (backen	pytest	Develover	Before opening a pull request for each implement feature
Functional Testing	Manual	QA	After adding a new feature manually test functionali
Integration testing	pytest+Postman /Playwright	QA	End Of each iteration
Regression (Front End)	Playwright+python	QA	-Before each demo, - for every pull request in github action

h. Defect Management

The project will use Jira to manage and track all defects. Defects may include functional bugs, UI issues, integration problems, and incorrect AI-generated outputs. Developers will be responsible for fixing defects in bugfix branches, while testers or team members will identify and report issues during testing and code reviews. Each defect will be logged in Jira and then assigned for resolution, reviewed through pull requests, tested, and closed after verification.

We will follow following defect severity level:

- Critical: Application crashes, core feature completely blocked
- High: A Major feature works incorrectly or produced incorrect output
- Medium: A feature works but behavior unexpectedly in certain ways
- Low: Edge cases. A feature doesn't work as expected in certain exact flow and the user flow is not normal

6. AI usage Log

(This table summarizes AI contribution in this document)

Section	Who	AI contribution (%)	Description	Verified by
1	Jannatul Tuba	1%	Used AI for brainstorming project ideas, exploring functional requirement ideas, and researching different tech stack options to consider all possible solutions and best practices	Danielle Leask

7. Project Links

Github link: https://github.com/BUMETCS673/cs673olsum26project-cs673olsum26team3
Project management tool link (e.g. Jira):

8. References

https://google.github.io/styleguide/htmlcssguide.html
https://google.github.io/styleguide/pyguide.html

9. Glossary

N/A at the moment